

Líneas de código necesarias ...PARA EMPEZAR A PROGRAMAR...

**Programación – 1º DAM
Curso 2024/2025**

Lectura de teclado

Para poder **leer por teclado** instanciaremos la clase Scanner de la siguiente forma:

- Creamos el objeto:

```
Scanner sc=new Scanner(System.in) ;
```

- En función del tipo de dato que queramos leer llamaremos a un método o a otro:

- Si es entero:

```
int num;  
num=sc.nextInt() ;
```

- Si es double:

```
double num;  
num=sc.nextDouble() ;
```

- Si es String:

```
String s;  
s=sc.next() ; //lee la primera palabra hasta que se encuentra un espacio  
s=sc.nextLine() ; //lee hasta que encuentra el primer salto de línea
```

- Si es char:

```
char c;  
c=sc.next().charAt(0) ;
```

Escritura por pantalla (consola)

Para **mostrar por consola**:

- Reenviaremos a la salida estándar el texto que queremos mostrar. Pondremos entre comillas el texto “literal” que queramos mostrar y concatenaremos con el operador + las variables para que muestren su valor:

```
System.out.println("Hola mundo"); //mostrará: Hola mundo
```

```
int numero=20;
```

```
System.out.println("El valor de número es "+numero);  
//mostrará: El valor de número es 20
```

Comparar valores de cadenas

Si queremos comparar una variable de tipo String con una cadena concreta, no podemos utilizar el comparador `==`, debemos hacerlo con el método `equals()`:

```
String cadena;
```

```
if(cadena.equals("hola mundo")) //comprueba si la variable  
                                //cadena vale hola mundo
```

Si queremos comprobar que sea distinto, negamos la expresión anterior:

```
String cadena;
```

```
if(!cadena.equals("hola mundo"))//comprueba si la variable  
                                //cadena es distinta de  
                                //hola mundo
```

Generar valores aleatorios

Java nos proporciona un método dentro de la **clase Math** que nos permite generar números aleatorios. Este **método** es **random()** y funciona de la siguiente forma:

Código Java	Número aleatorio generado
Math.random()	Tipo double entre 0.0 y 1.0
(int) (Math.random() * (N+1))	Tipo int entre 0 y N
(int) (MIN + Math.random() * (MAX - MIN + 1))	Tipo int entre MIN y MAX

Tratamiento fechas

Obtener fecha actual sistema

```
LocalDateTime hoy = LocalDateTime.now();
```

Con el objeto LocalDateTime podemos acceder a métodos que nos devuelven el día, el mes o el año:

```
hoy.getDayOfMonth();  
hoy.getMonthValue();  
hoy.getYear();
```

estos tres métodos devuelven un valor entero (int).

Problemas con Scanner

Cuando usas **Scanner.nextInt()** (o **nextDouble()**, **next()...**), solo lee el número, pero NO consume el salto de línea (\n) que se produce cuando el usuario presiona Enter. Entonces, cuando luego llamas a **nextLine()**, este lee ese salto de línea pendiente como si fuera una entrada vacía, y no espera que escribas nada, porque ya tiene el \n en el buffer. Ejemplo en el que se produciría el fallo:

...

```
System.out.println ("Introduce tu edad:");  
int edad=sc.nextInt();  
System.out.println ("Introduce tu nombre:");  
String nombre=sc.nextLine();
```

Problemas con Scanner

No nos dejaría pasar el nombre porque cogería el salto de línea que provoca `nextInt()` pero no consume. Es decir, nombre tendría como valor `\n` (salto de línea).

La solución más sencilla y habitual es:

Después de usar `nextInt()`, agrega un `scanner.nextLine()`; solo para consumir el salto de línea restante. Ejemplo con esta solución:

```
System.out.println ("Introduce tu edad:");  
int edad=sc.nextInt();  
sc.nextLine(); //esto limpia el salto de línea que quedó  
System.out.println ("Introduce tu nombre:");  
String nombre=sc.nextLine(); //ahora sí permite pasarlo
```